

带分摊查询的高吞吐量原子存储链式复制

简介

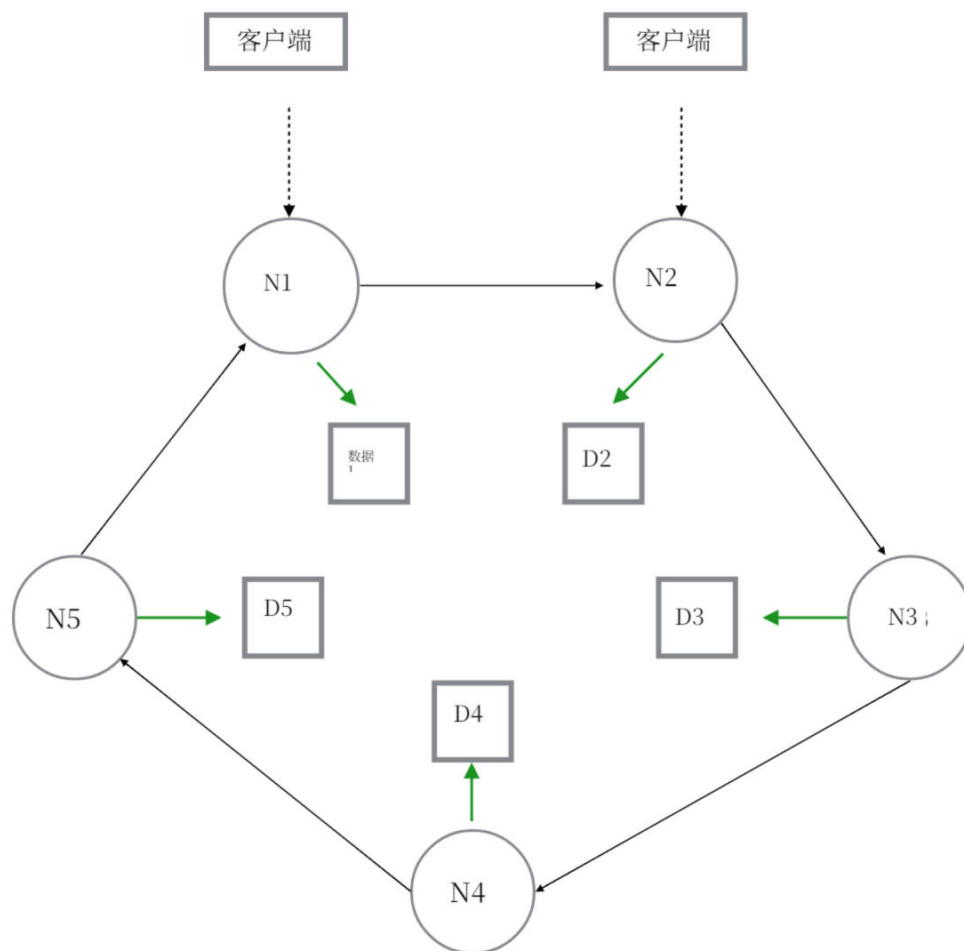
它是一个原子性的、高吞吐量且具备弹性的分布式存储系统。即便服务器出现故障，它也能始终保持一致性和可用性（CA）。

它假定所有服务器之间始终可以进行点对点通信（分区不兼容）。系统设置了故障检测机制来检测任何服务器故障，并且只要至少有一台服务器可用，系统就会继续运行。

客户端可以使用任何服务器进行并发读写，因此它是一个高吞吐量存储。它防止对同一对象进行并发更新以保持一致性。

它解决了读反问题，并防止在较早的读操作返回新值后，任何读操作返回旧值。

架构



$N1 \cdots Ni$ 是系统服务器，它们维护实现 CRAQ 功能所需的所有状态信息。

$D1 \cdots Di$ 是数据服务器，它们有助于数据的持久化。这是一个无模式、面向列的数据存储。

客户端可以通过给定的 API 与任何系统服务器进行交互。

系统服务器与本地数据服务器进行交互，以实现数据的查询与持久化。

系统服务器呈环形排列，在预写阶段，它们与后继服务器（顺时针方向）进行通信，而在写入阶段，则与前驱服务器（逆时针方向）通信。

在这个实现中，没有固定的头 / 尾节点，但是对于每一条更新消息，发起者节点成为头节点，发起者节点的前驱节点成为尾节点，因此客户端可以使用任何节点进行更新。

更新消息的发起（头部）节点启动预写阶段，并向其顺时针方向的后继节点发送预写消息。最终，预写消息将到达发起节点的前驱（尾部）节点。一旦发起节点的前驱节点收到预写消息，它就会启动写入阶段，并向其逆时针方向的前驱节点发送写入消息，最终写入消息将返回发起节点（头部）。一旦发起节点接收到写入消息，它就会向客户端回复有关更新的信息。

它还支持小型事务。客户端可以在一次调用中更新多个对象。如果发生写冲突，整个事务将被中止。

用户可以将系统配置为使用 lager 日志框架或 gen_event/OTP 日志框架。

写冲突

如果有两个或更多节点试图更新同一对象，就会发生写冲突。如果同一对象是迷你事务并发更新的一部分，那么只允许一个更新 / 迷你事务完成，其余的更新 / 迷你事务将被中止。

如果所有并发更新都处于预写阶段，那么具有最高发起（头部）节点值的更新消息将获胜。

如果并发更新同时存在于预写阶段和写入阶段（写入阶段只能有一个更新），那么写入阶段的更新消息优先。

数据检查点与数据文件版本控制

在进行若干更新记录后，会检查当前数据文件是否存在数据损坏。如果未发现数据损坏，当前数据文件将被关闭，并创建新的带版本的数据文件。如果发现任何数据损坏，当前数据文件将在正确的位置被截断，并且该 CRAQ 节点将从环中移除。用户需要使用以下方法将该 CRAQ 节点重新添加到环中。

添加节点函数。一旦该 CRAQ 节点重新添加回来，它会根据丢失的数据量从其后继节点获取增量快照，然后再次变为最新状态。

sys. config

replication_type	: It is only for documentation purpose. It is not being used.
node_order	: It takes a value sorted or user_defined sorted means the replicated ring is created using the sorted list of node values user_defined means, user provides the ring order
failure_detector	: Name of failure detection module that sets up the failure detection feature
repl_data_manager	: Name of module for interacting with data_server
storage_data	: Name of module which decides the file format of data where data_server persists data
write_conflict_resolver	: Name of module used for write conflict resolution
unique_id_generator	: Name of module used to generate unique id for any async request
query_handler	: Name of the module that is being used to handle a query request for an object if that object is in <i>pre-write</i> phase. One can use any of the following modules: eh_wait_query_handler_api system_server will wait for new value to persist and only then it will respond to the client eh_no_wait_query_handler_api system will not wait and respond immediately saying that the object is being updated eh_dirty_read_query_handler_api system_server will respond immediately with existing old value (<i>dirty_read</i>) eh_aq_query_handler_api query request will be send to <i>tail</i> to respond.
event_logger	: User can configure the system to user either lager logging framework (assign value lager_event) or gen_event/OTP logging framework (assign value gen_event).
data_checkpoint	: system will check the data file for data corruption after the number of update entries specified through this parameter. If the the data file is not corrupted then system will close the current data file and create a new versioned data file.
data_dir	: path where data and log files will be persisted if left blank then data and log files will be persisted in erlang_htas home directory (explained later)
file_repl_data	: this will be added as suffix to the node name to create actual data file name
file_repl_data_suffix	: when a versioned data file is created, this parameter is used to create a padded data file version number that is appended to file name.

<code>file_repl_log</code>	: this can be either set to <code>standard_io</code> or a file suffix. If this value is set to <code>standard_io</code> then all the log messages will be displayed on console otherwise they will be written in file with file name where this value will be suffix to node name
<code>debug_mode</code>	: log messages will appear only when this value is set to true.
<code>sup_restart_intensivity</code> <code>sup_restart_period</code> <code>sup_child_shutdown</code>	: these 3 parameters are used by OTP supervisor

上述出现的所有模块名称都是基于行为命名的，如果有人希望以不同方式处理这些功能，这些名称可以更改。

设置

使用 `craq_erl.sh` 启动一个 Erlang 分布式节点。此脚本使用短名称（`sname`）来创建分布式节点，但如果短名称不适合你，你可以将其更改为长名称（`name`）。

为 `erlang_craq` 创建一个主目录。

在这个主目录下创建一个名为 `ebin` 的子目录（文件夹），并将所有的 `*.beam` 和 `*.app` 文件复制到这里。

在这个主目录下创建一个子目录（文件夹）`craq_test/ebin`，并复制所有用于自动化测试的 `*.beam` 文件。

修改这个 Bash 脚本的第一行，以便进入你的 `erlang_craq` 主目录。

使用 `./craq_erl.sh <sname>` 创建一个 Erlang 分布式节点，然后使用以下命令启动 `erlang_craq` 服务器。

```
erlang_craq:start().
```

如果您想关闭 `erlang_craq` 服务器（应用程序），那么您可以在每个分布式 Erlang 服务器节点上输入以下命令。

```
erlang_craq:stop().
```

`erlang_craq:start()` 只会创建 1 个 `erlang_craq` 服务器，通常你需要不止 1 个服务器，因此请在不同的终端窗口中重复此步骤以创建更多服务器。

创建 3 个节点的 `erlang_craq` 分布式节点的一个典型示例是在 3 个不同的终端中按如下方式执行以下命令：

```
terminal 1 :  ./craq_erl.sh  ec_n1
> erlang_craq:start().
```

```
terminal 2 :  ./craq_erl.sh  ec_n2
> erlang_craq:start().
```

```
terminal 3 :  ./craq_erl.sh  ec_n3  
> erlang_craq:start().
```

ERLANG_CRAQ 应用程序编程接口

我们将对象定义为对象类型和对象 ID 的组合。例如 {person, 10}。

在这里的大多数示例中，我们将使用原子和整数，但它们可以是任何 Erlang 术语。

所有的 erlang_craq API 命令都从一个分布式客户端节点执行。因此，使用 craq_erl.sh bash 脚本创建一个新的分布式客户端 erlang 节点（不要在客户端节点上启动任何 erlang 服务器）。

为客户端创建分布式节点的一个典型示例如下：

```
终端 4: ./craq_erl.sh ec_c1
```

您可以从这个客户端节点执行以下任何命令。

设置交互式环境

erlang_craq:setup_repl (节点列表)。

这是在你进行任何有意义的操作之前要执行的第一条命令。该命令会将你之前创建的所有 Erlang 服务器连接成一个环，并设置故障检测。

在这里，NodeList 是一个你之前创建的 Erlang 分布式节点列表。 以我的电脑为例）典型的 NodeList 如下：

```
N1="eh_n1@Gyanendras-MacBook-Pro".  
N2="eh_n2@Gyanendras-MacBook-  
Pro".N3="eh_n3@Gyanendras-MacBook-Pro".
```

节点列表 = [N1, N2, N3]。

更新

此命令用于插入新值或更新现有值。

erlang_craq:update (节点, 对象类型, 对象 ID, [{列 1, 值 1}, ..., {列 n, 值 n}])。

此命令将向节点发送并更新消息。

示例

```
erlang_craq:update(N1, person, 10, [{name, john}, {age, 30}, {gender, male}]).
```

如果你想更新 {person, 10} 的年龄，可以在同一个节点或环中的任何其他节点上执行以下命令。

```
erlang_craq:update(N2, person, 10, [{age, 40}]).
```

你还可以使用此命令在单个命令中更新和删除列。其格式如下：

erlang_craq:update (节点, 对象类型, 对象 ID, 更新列列表, 删除列列表)。

示例

我们想将名字更新为 john_smith 并删除年龄，所以我们的命令如下：

```
erlang_craq:update(N3, person, 10, [{name, john_smith}], [age]).  
(将节点 N3 上的 person 对象, ID 为 10 的记录, 把 name 更新为 john_smith, 并删除 age 列。)
```

我们可以在单个更新命令中执行小型事务（更新多个对象）

erlang_craq:update (节点, 更新列表)。

示例

```
erlang_craq:update(N1, [{candidate, 10, [{name, donald_trump},  
{party, republican}]], {person, 10, [{name, john_smith}, {age, 40}]}]).
```

删除

此命令用于从对象中删除列或删除整个对象。

删除列

erlang_craq:delete (节点, 对象类型, 对象 ID, 删除列列表)。

示例

```
erlang_craq:delete(N1, person, 10, [gender]). 这句话的意思是：在 erlang_craq 模块  
中，使用 delete 函数，在节点 N1 上，从类型为 person、标识为 10 的对象中删除 gender  
列。
```

删除整个对象

erlang_craq:delete (节点, 对象类型, 对象 ID)。

示例

```
erlang_craq:delete(N1, person, 10).
```

查询

此命令用于查询对象。

erlang_craq:query (节点, 对象类型, 对象 ID)。

示例

```
erlang_craq:query(N1, person, 10).
```

停止

此命令主要是为测试目的而创建的。除此之外，绝不应该使用它。此命令用于从客户端节点关闭服务器。服务器确实会关闭。由于它在 OTP 监管下运行，因此监管程序会将其重启。服务器确实会重启，但它不会自动成为集群的一部分，所以实际上，对于集群而言，它仍然处于关闭状态。

erlang_craq:stop (Node). 翻译为: erlang_craq:
停止 (节点)。

此命令将使节点停机。

示例

```
erlang_htas: 停止 (N1)。
```

添加节点

此命令用于将节点重新添加到环中。

erlang_craq:add_node (节点, 节点列表, 节点顺序列表)。

NodeList 应包含所有节点的列表（包括正在添加的节点）。NodeOrderList 是所有节点的列表，包括任何下级节点。如果节点顺序已定义为用户自定义，则此列表必须按照正确的节点顺序排列。

最有可能的是，就其所包含的数据而言，添加到环中的节点在环中会落后于其他节点。因此，新添加的节点总是从其后继节点获取增量数据快照，以保持最新状态。

示例

```
erlang_craq:add_node(N1, [N1, N2], [N1, N2, N3])。
```

手动测试

从 `data_dir` 或 `erlang_craq` 目录中删除所有 `.data` 文件，以确保我们在一致的状态下执行此测试。

按照以下方式启动 3 个 erlang_craq 服务器和一个客户端节点。

```
terminal 1 : ./craq_erl.sh   ec_n1
             erlang_craq:start().
```

```
terminal 2 : ./craq_erl.sh   ec_n2
             erlang_craq:start().
```

```
terminal 3 : ./craq_erl.sh   ec_n3
             erlang_craq:start().
```

```
terminal 4 : ./craq_erl.sh   ec_c1
```

现在在 4 号终端（它是一个客户端节点）上，按如下方式创建一个节点列表。

```
> N1 = 'ec_n1@<your_host_name>'.
> N2 = 'ec_n2@<your_host_name>'.
> N3 = 'ec_n3@<your_host_name>'.
> NL = [N1, N2, N3].
```

设置复制环

```
| erlang_craq:setup_repl(NL).
```

使用 N1 插入数据

```
| erlang_craq:update(N1, candidate, 10, [{name,donald_trump},{party,repUBLICan}]).
```

并且你应该看到如下响应：

```
[[ok, [{candidate, 10}], updated]]
```

查询 N1、N2 和 N3，确保数据已插入到所有 3 个节点中。

```
> erlang_craq:query(N1, candidate, 10).
> erlang_craq:query(N2, candidate, 10).
> erlang_craq:query(N3, candidate, 10).
```

并且你应该看到每个查询的如下响应：

```
[[ok, {candidate, 10, [{party, repUBLICan}, {name, donald_trump}]}]]
```

让我们创建更多更新条目

```
> erlang_craq:update(N2, candidate, 20, [{name,hillary_clinton},{party, democrat}]).
> erlang_craq:update(N3, candidate, 30, [{name,ted_cruz},{party,repUBLICan}]).
```

一笔小额交易


```
> erlang_craq:update(N1, [{candidate, 40, [{name,bernie_sanders},{party, democrat}]},  
                           {candidate, 50, [{name,marco_rubio},{party, republican}]}]).
```

现在，让我们对不同对象进行并发的微事务更新，这可以通过 multi_update API 来完成。

此 API 用于模拟单个客户端的并发更新。

```
> erlang_craq:multi_update([N1, N2, N3],  
                           [{candidate, 10, [{name,donald_trump},{party, republican}]},  
                            {candidate, 20, [{name,hillary_clinton},{party, democrat}]}],  
                           [{candidate, 30, [{name,ted_cruz},{party, republican}]},  
                            {candidate, 40, [{name,bernie_sanders},{party, democrat}]}],  
                           [{candidate, 50, [{name,marco_rubio},{party, republican}]},  
                            {candidate, 60, [{name,ben_carson},{party, republican}]}]).
```

这两个更新都应该成功，并且你应该看到以下响应：

```
{ok, [{candidate, 10}, {candidate, 20}], updated},  
{ok, [{candidate, 30}, {candidate, 40}], updated},  
{ok, [{candidate, 50}, {candidate, 60}], updated}}
```

让我们对同一个对象进行并发更新。

```
> erlang_craq:multi_update([N1, N2, N3],  
                           [{candidate, 10, [{name,donald_trump},{party, republican}]},  
                            {candidate, 20, [{name,hillary_clinton},{party, democrat}]}],  
                           [{candidate, 30, [{name,ted_cruz},{party, republican}]},  
                            {candidate, 20, [{name,hillary_clinton},{party, democrat}]}],  
                           [{candidate, 50, [{name,marco_rubio},{party, republican}]},  
                            {candidate, 20, [{name,hillary_clinton},{party, democrat}]}]).
```

它应该只允许其中一个更新操作成功，因此你应该会看到以下响应。

```
{error, [{candidate, 10}, {candidate, 20}], being_updated},  
{error, [{candidate, 30}, {candidate, 20}], being_updated},  
{ok, [{candidate, 50}, {candidate, 20}], updated}}
```

现在，让我们停止 N1，使用 N2 和 N3 进行更多更新。

```
> erlang_craq:stop(N1).
```

```
> erlang_craq:update(N3, candidate, 80, [{name,chris_christie},{party, republican}]).  
> erlang_craq:update(N2, candidate, 90, [{name,jeb_bush},{party, republican}]).
```

在这个阶段，检查数据文件的文件大小，你会发现 ec_n1_repl.data 比 ec_n2_repl.data 和 ec_n3_repl.data 小。ec_n2_repl.data 和 ec_n3_repl.data 的大小应该相同。现在，让我们恢复 N1。

```
> erlang_craq:add_node(N1, NL, NL).
```

现在，再次检查文件大小，你会发现所有 3 个数据文件的大小相同。另外，为确保 N1 有候选人 80 和候选人 90 的数据，请执行以下命令。

```
> erlang_craq:query(N1, candidate, 80).  
> erlang_craq:query(N1, candidate, 90).
```

自动化测试

要使用这个自动化测试工具，我们需要多个客户端。

一个客户端将每隔 5 秒随机关闭或启动一个 erlang_craq 服务器。该客户端将执行此操作约 15 到 20 次，如果在运行结束时，仍有一些节点处于关闭状态，它将逐个将其恢复。

其他客户端（至少还有 2 个）将持续输入数据。它们将以每秒 5 条的速度进行大约 400 到 500 条输入。它们会从节点列表中随机选择一个节点来输入数据。如果该节点出现故障也没关系，因为客户端对该数据输入会超时。客户端还会为一条数据输入随机选择一个对象类型、对象 ID、列名和列值。一旦所有客户端完成工作，我们就可以验证我们的 Erlang_Craq 服务器是否处于一致状态。这些是执行我们的自动化测试所需的步骤。

- 删除所有数据文件
- 启动 6 个终端，并按照以下给出的说明操作。

```
terminal 1 : ./craq_eri.sh ec_n1  
> erlang_craq:start().
```

```
terminal 2 : ./craq_eri.sh ec_n2  
> erlang_craq:start().
```

```
terminal 3 : ./craq_eri.sh ec_n3  
> erlang_craq:start().
```

```
terminal 4 : ./craq_eri.sh ec_c1  
> NL = ['ec_n1@<host_name>', 'ec_n2@<host_name>', 'ec_n3@<host_name>'].
```

现在准备好以下命令，但不要执行它
| erlang_craq_test:data_entries(NL).

```
terminal 5 : ./craq_eri.sh ec_c2  
> NL = ['ec_n1@<host_name>', 'ec_n2@<host_name>', 'ec_n3@<host_name>'].
```

现在准备好以下命令，但不要执行它
| erlang_craq_test:data_entries(NL).

```
terminal 6 : ./craq_erl.sh ec_c3  
> NL = ['ec_n1@<host_name>', 'ec_n2@<host_name>', 'ec_n3@<host_name>'].
```

现在在这个终端上执行以下命令。

```
> erlang_craq:setup_repl(NL).
```

```
> erlang_craq_test:node_change(NL).
```

现在，回到 4 号终端和 5 号终端，执行 data_entries 命令。

一旦所有 3 个客户端完成工作，在终端 6 上执行以下命令，以验证 erlang_htas 服务器是否处于一致状态。

```
> erlang_craq_validate:check_data(NL).
```

并且你应该看到如下响应

有效的

任何其他响应都将表明系统未通过测试。